Como instalar e rodar o projeto (API de Produtos)

Este projeto implementa uma API em **Node.js + Express + MySQL**, com CRUD completo da entidade **produtos**.

Link do video com o backend rodando no Youtube: https://youtu.be/xunUL5jhAis

1. Programas necessários

Instale estes programas antes de começar:

1. **Visual Studio Code**
2. **Node.js**
3. **Postman**
4. **MySQL Server + MySQL Workbench**
    a. *O MySQL SERVER precisa estar instalado e rodando.*
5. **XAMPP** (abra o xampp e clique em start nas duas primeiras opçoẽs)

2. Abrir o projeto no VS Code

1. Baixe o arquivo **Rotas.zip**
2. Extraia a pasta
3. Abra o **VS Code**
4. Vá em **File** → **Open Folder**
5. Selecione a pasta **Rotas**

3. Testar se o Node está funcionando

1. No VS Code:
2. Menu → Terminal → New Terminal
3. Agora digite: cmd (o código rodará dentro do command ao invés do powershell)
4. Agora digite:

5. node -v

→ Se aparecer um número com a versão, está ok.

4. Instalar dependências do projeto

Ainda no terminal:

npm install
npm start
Npm run dev

Possível problemas:

npm error code ENOENT
npm error syscall open
npm error path C:\Users\25028157\Downloads\Rotas\package.json
npm error errno -4058
npm error enoent Could not read package.json: Error: ENOENT: no such file or directory, open 'C:\Users\25028157\Downloads\Rotas\package.json'
npm error enoent This is related to npm not being able to find a file.
npm error enoent
npm error A complete log of this run can be found in: C:\Users\25028157\AppData\Local\npm-cache_logs\2026-03-30T22_21_02_094Z-debug-0.log

Como resolver:

Possivelmente ao baixar o arquivo a pasta for duplicada.
Para resolver rode o comando: cd rotas

Agora prossiga com os comandos:
npm install
npm start

5. Configurar o arquivo .env

Na raiz do projeto, certifique-se de que o arquivo .env contém:

PORT=3000
MYSQL_HOST=localhost
MYSQL_USER=root
MYSQL_PASSWORD=**SUA_SENHA_AQUI**
MYSQL_DB=aula_backend

Se sua senha MySQL for vazia, deixe assim:

MYSQL_PASSWORD=

6. Iniciar o servidor

No terminal, dentro da pasta Rotas, digite:

npm run dev

Você deve ver:
Servidor On em http://localhost:3000

7. Configurar o banco de dados no MySQL Workbench

Abra o **MySQL Workbench**.

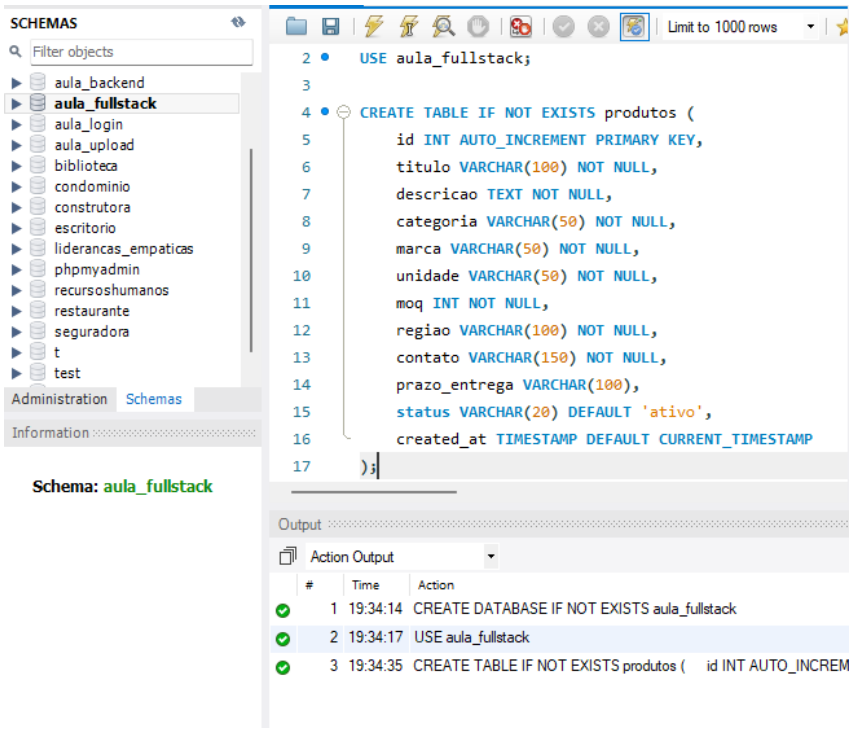
Crie uma nova conexão se necessário (Local Instance MySQL80).

Depois, cole e execute o seguinte:

```
CREATE DATABASE IF NOT EXISTS aula_fullstack;

USE aula_fullstack;


CREATE TABLE IF NOT EXISTS produtos (

    id INT AUTO_INCREMENT PRIMARY KEY,

    titulo VARCHAR(100) NOT NULL,
```

```
    descricao TEXT NOT NULL,
    categoria VARCHAR(50) NOT NULL,
    marca VARCHAR(50) NOT NULL,
    unidade VARCHAR(50) NOT NULL,
    moq INT NOT NULL,
    regiao VARCHAR(100) NOT NULL,
    contato VARCHAR(150) NOT NULL,
    prazo_entrega VARCHAR(100),
    status VARCHAR(20) DEFAULT 'ativo',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

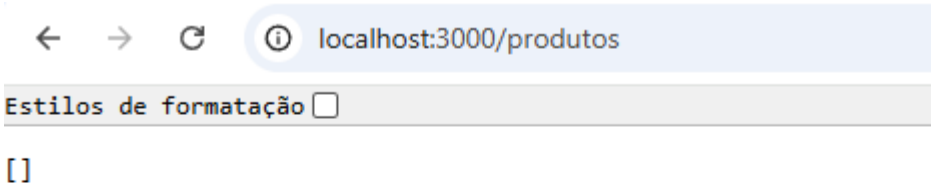
Certifique-se que seu banco de dados esteja conforme abaixo:



8. Testar no navegador

Com o servidor rodando, abra no navegador: http://localhost:3000/produtos

Se aparecer: [ ]
Significa que a API está funcionando corretamente e o banco está vazio.



9. Testar no Postman

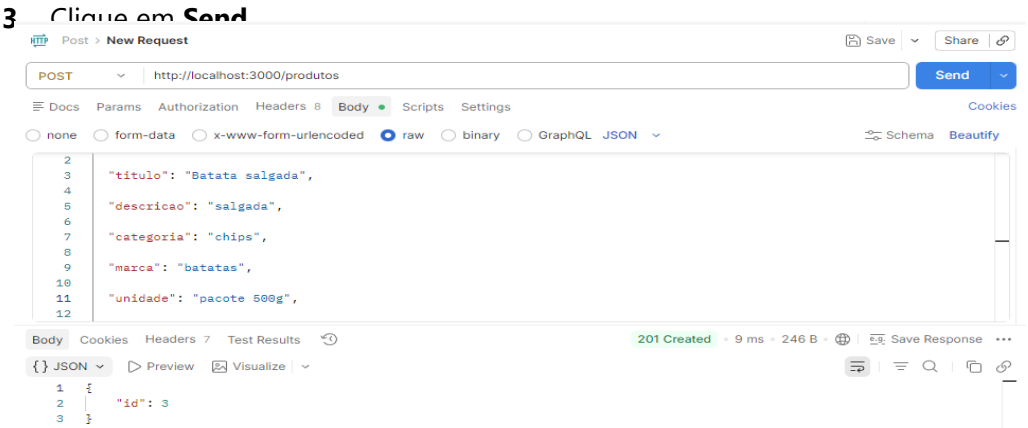
Criar um produto (POST)

- No Postman, crie uma nova requisição
- Método **POST**
- URL: http://localhost:3000/produtos

1. Vá em **Body** → **raw** → **JSON**
2. Cole:

```
{
  "titulo": "Batata Chips",
  "descricao": "Sabor churrasco",
  "categoria": "chips",
  "marca": "MrNut",
  "unidade": "pacote 50g",
  "moq": 50,
  "regiao": "SP",
  "contato": "email@empresa.com"
}
```

3. Clique em **Send**



Atualizar o produto (PUT)

Agora que o produto 1 existe, vamos atualizar.

1. No Postman, mude o método para **PUT**
2. URL:http://localhost:3000/produtos/1

1. Body → raw → JSON
2. Cole:


```
{
  "titulo": "Batata Chips",
  "descricao": "Sabor churrasco",
  "categoria": "chips",
  "marca": "MrNut",
  "unidade": "pacote",
  "moq": 20,
  "regiao": "SP",
  "contato": "email@exemplo.com"
}
```


Clique em **Send**

Deve retornar:


```
{
"message": "Produto atualizado com sucesso"
}
```


Buscar produto por ID (GET)

1. No Postman, mude o método para **GET**
2. URL: http://localhost:3000/produtos/1

Resposta esperada:


```
{
  "id": 1,
```

"titulo": "Batata Chips",
"descricao": "Sabor churrasco",
"categoria": "chips",
"marca": "MrNut",
"unidade": "pacote",
"moq": 20,
"regiao": "SP",
"contato": "email@exemplo.com",
"prazo_entrega": null,
"status": "ativo",
"created_at": "2026-03-28T00:59:25.000Z"
}

Listar todos os produtos (GET)

1. Método: **GET**
2. URL: http://localhost:3000/produtos

Resposta esperada:

[
{
"id": 1,
"titulo": "Batata Chips",
"descricao": "Sabor churrasco",
"categoria": "chips",
"marca": "MrNut",
"unidade": "pacote",
"moq": 20,
"regiao": "SP",
"contato": "email@exemplo.com",
"prazo_entrega": null,
"status": "ativo",
"created_at": "2026-03-28T00:59:25.000Z"
}
]

Deletar um produto (DELETE)

1. No Postman, mude o método para **DELETE**
2. URL: http://localhost:3000/produtos/1

Clique em Send

Resposta esperada:

```
{
  "message": "Produto removido com sucesso"
}
```